

Chapter 12

조인 튜닝1 풀이

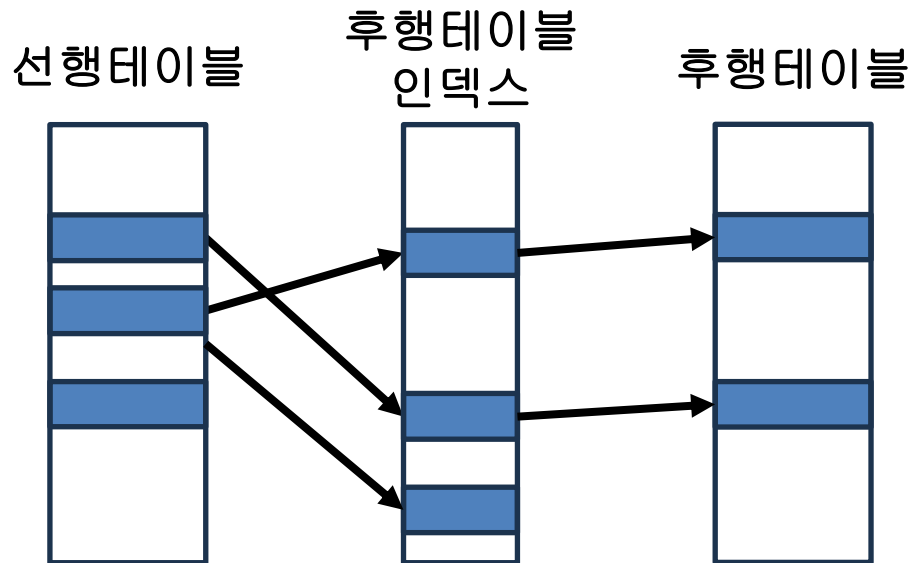
1. Oracle에서 TAB1,TAB2 순으로 NL 조인하도록 유도하고 싶다. 다음 중 (가)안에 넣을 바른 힌트 사용법을 2개 고르시오.

```
SELECT /*+(          가          ) */ .....  
FROM TAB1 A, TAB2 B  
WHERE A, KEY = B, KEY
```

- ① ORDERED USE_NL(B)
- ② ORDERED USE_NL(TAB2)
- ③ LEADING(A) USE_NL(B)
- ④ DRIVING_SITE(A) USE_NL(B)

NL 조인 (Nested Loop Join)의 개념

선행 테이블(Driving Table)의 결과 기준으로 후행테이블로 순차적으로 연결해 데이터를 조인하는 기법



- 프로그램의 반복문을 중첩시킨 구조랑 유사한 방법으로 조인
- 선행 테이블을 얼마나 적게 조회하는지가 성능에 영향을 미침
- 후행 테이블이 인덱스가 없을 경우 심각한 성능저하가 발생

NL 조인 (Nested Loop Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
Select /*+ ordered use_nl(B) */ A.EMPNO, A.DEPTNO, B.DNAME
from emp A
      , dept B
where A.deptno = B.deptno
AND A.deptno = 10;

select plan_table_output from table(dbms_xplan.display(null, null, 'serial'));
```

- USE_NL, ordered 힌트를 통해서 실행계획 제어가 가능
- **USE_NL** : NL 방식 조인 사용
- **ordered** : FROM 절에 기술된 순서대로 조인

NL 조인 (Nested Loop Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
select /*+ leading(A, B, C) use_nl(B) use_nl(C) */
      A.EMPNO, A.DEPTNO, B.DNAME, C.EMPNO
from emp A, dept B, emp C
where A.deptno = B.deptno AND A.deptno = 10 AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null, null, 'serial'));
```

– 3개 이상 테이블 조인 시 **leading** 힌트를 통해 조인 순서를 제어

```
select /*+ use_nl(A, B, C) */
      A.EMPNO, A.DEPTNO, B.DNAME, C.EMPNO
from emp A, dept B, emp C
where A.deptno = B.deptno AND A.deptno = 10 AND A.MGR = C.EMPNO;
```

– ordered 나 leading을 사용하지 않으면 옵티마이저가 조인 순서를 결정

① ORDERED USE_NL(B) (O)

② ORDERED USE_NL(TAB2) (X)

→ TAB2는 "B"로 Alias 되어 있어 "B"로 입력해야지 힌트가 동작

③ LEADING(A) USE_NL(B) (O)

④ DRIVING_SITE(A) USE_NL(B) (X)

→ DRIVING_SITE은 분산환경(DB Link)에서 원격지로 JOIN을 할 수 있게 하는 힌트

DB Link 표기 필요 (TAB1@ora1 A)

→ ORDERED 또는 LEADING 와 같이 조인순서 명시하는 것이 NL 조인을 유도하는데 유리함

(정답) ①, ③

2. 아래 SQL을 힌트에 지시한 대로 수행할 때, 조건절 비교 순서를 올바르게 나열한 것을 고르시오.

[인덱스 구성]

사원_PK : 사원번호

사원_X1 : 입사일자

고객_PK : 고객번호

고객_X1 : 관리사원번호

```
select /*+ ordered use_n1(c) index(e) index(c) */
```

e. 사원번호, e. 사원명, e. 입사일자, c.고객번호, c. 고객명, c.전화번호, c.최종주문금액

from 사원 e, 고객 c

where c. 관리사원번호 = e. 사원번호 -----가

and e. 입사일자 >= '20200101' -----나

and e.부서코드 = '10010' -----다

and c. 최종주문금액 >= 20000 -----라

① 가 → 나 → 다 → 라

② 나 → 다 → 가 → 라

③ 나 → 다 → 라 → 가

④ 나 → 가 → 다 → 라

NL 조인 (Nested Loop Join)의 실행계획 해석

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	59	4 (25)	00:00:01
1 ④	SORT ORDER BY		1	59	4 (25)	00:00:01
2	NESTED LOOPS		1	59	3 (0)	00:00:01
3 ②	TABLE ACCESS BY INDEX ROWID	DEPT	1	20	1 (0)	00:00:01
* 4 ①	INDEX UNIQUE SCAN	PK DEPT	1		0 (0)	00:00:01
* 5 ③	TABLE ACCESS FULL	EMP	1	39	2 (0)	00:00:01

Predicate Information (identified by operation id):

4 - access("B"."DEPTNO"=10)
 5 - filter("A"."DEPTNO"=10 AND "A"."SAL">1000)

- 조인 방법으로 하위의 테이블들이 Nested Loop로 조인
- ① DEPT 테이블의 PK_DEPT Object로 조회
- ② ROWID로 테이블을 Access
- ③ EMP 테이블을 FULL Scan
- ④ Nested Loop Join 후 SORT ORDER BY 실행

```
select /*+ ordered use_n1(c) index(e) index(c) */
      e. 사원번호, e. 사원명, e. 입사일자, c.고객번호, c. 고객명,
      c.전화번호, c.최종주문금액
from 사원 e, 고객 c
where c. 관리사원번호 = e. 사원번호 -----가
and   e. 입사일자 >= '2020101' -----나
and   e.부서코드 = '10010' -----다
and   c. 최종주문금액 >= 20000 -----라
```

- ① ordered 힌트로 사원 테이블을 먼저 액세스
- ② 사원테이블의 index 인 사원_X1(입사일자)로 액세스 (나)
- ③ 인덱스로 액세스한 결과로 사원테이블을 액세스하면서
부서코드 = '10010'으로 필터링 (다)
- ④ NL 조인을 위해서 고객 테이블과 JOIN (가)
- ⑤ NL 조인을 할때 고객 테이블을 액세스하면서
최종주문금액 >= 20000 으로 필터링 (라)

(정답) ② 나 → 다 → 가 → 라

3. 다음 중 아래와 같은 SQL에서 가능한 조인 방법을 모두 나열한 것은?

```
SELECT A.ENAME, B.GRADE  
FROM EMP A, SALGRADE B  
WHERE A.SAL BETWEEN B.LOWER_SAL AND B.HIGH_SAL
```

- ① Nested Loop Join
- ② Nested Loop Join, Sort Merge Join
- ③ Sort Merge Join, Hash Join
- ④ Nested Loop Join, Sort Merge Join, Hash Join

① Nested Loop Join (X)

→ Sort Merge Join 도 가능

② Nested Loop Join, Sort Merge Join (O)

③ Sort Merge Join, Hash Join (X)

→ Hash Join은 등치(=) 조인만 가능

④ Nested Loop Join, Sort Merge Join, Hash Join (X)

→ Hash Join은 등치(=) 조인만 가능

(정답) ①, ③

4. 아래 SQL은 index 힌트에 인덱스명이나 컬럼명을 지정하지 않았으므로 여러 가지 액세스 경로가 가능하다. 보기는 각 액세스 경로에서 사용하게 될 인덱스를 나열한 것인데, 가장 가능성이 작은 것은?

[인덱스 구성]

사원_PK : 사원번호

사원_X1 : 입사일자

사원_x2 : 부서코드

고객_PK : 고객번호

고객_X1 : 관리사원번호

고객_X2 : 최종주문금액

```
select /*+ ordered use_nl(c) index(e) index(c) */
```

e. 사원번호, e. 사원명, e. 입사일자, c.고객번호, c. 고객명, c. 전화번호, c. 최종주문금액

```
From 사원 e, 고객 c
```

```
where c. 관리사원번호 = e. 사원번호
```

```
and e. 입사일자 >= '20200101'
```

```
and e.부서코드= 10010'
```

```
and c.최종주문금액 >= 20000
```

① 사원_PK, 고객_X1

② 사원_X1, 고객_X1

③ 사원_X1, 고객_X2

④ 사원_X2, 고객_X1

조건절 별 사용 예상되는 인덱스

where c. 관리사원번호 = e. 사원번호

→ 고객_X1

and e. 입사일자 > = '20200101'

→ 사원_X1

and e.부서코드= 'Z123'

→ 사원_X2

and c.최종주문금액 > = 20000

→ 고객_X2

해설

```
select /*+ ordered use_n1(c) index(e) index(c) */  
      e. 사원번호, e. 사원명, e. 입사일자, c.고객번호, c. 고객명, c.전화번호, c.최종주문금액  
from 사원 e, 고객 c  
where c. 관리사원번호 = e. 사원번호  
and   e.입사일자 >= '2020101'  
and   e.부서코드 = 10010'  
and   c. 최종주문금액 >= 20000
```

- ① ordered 힌트로 사원 테이블을 먼저 액세스
- ② 사원테이블의 index 인 **사원_X1**(입사일자)로 스캔
또는
사원테이블의 index 인 **사원_x2**(부서코드)로 스캔
- ③ NL 조인을 위해서 고객 테이블과 JOIN, 이때 **고객_X1**(관리사원번호)로 스캔
하거나
고객 테이블의 index 인 **고객_X2**(최종주문금액)로 스캔

조합1 : 사원_X1, 고객_X1

조합1 : 사원_X1, 고객_X2

조합3 : 사원_X2, 고객_X1

조합4 : 사원_X2, 고객_X2

(정답) ① 사원_PK, 고객_X1

5. 아래 SQL에 대한 튜닝 방안으로 가장 적절한 것을 고르시오.

[인덱스 구성]

상품_PK : 상품코드

상품_X01 : 상품분류코드 + 상품가격 + 공급업체코드

거래_PK : 거래번호

거래_X01 : 거래일자 + 상품코드

거래_X02 : 상품코드 + 거래구분코드 + 거래일자

```
select p. 상품코드, p. 상품가격, t. 거래일자, t. 거래수량, t. 거래금액
from 상품 p, 거래 t
where p. 상품분류코드 = 'KTG'
and p. 공급업체코드 = 'SP83732'
and p. 상품가격 between 10000and 100000
and t. 상품코드 = p. 상품코드
and t.거래일자 between ' 20210101' and ' 20210131'
```

Rows	Row Source Operation
------	----------------------

368	NESTED LOOPS (cr=1311 pr=124 pw=0 time=1622 us)
69	TABLE ACCESS BY INDEX ROWID 상품 (cr=986 pr=35 pw=0 time=659 us)
69	INDEX RANGE SCAN 상품_X01 (cr=922 pr=31 pw=0 time=645)
368	TABLE ACCESS BY INDEX ROWID 거래 (cr=325 pr=89 pw=0 time=963 us)
385	INDEX RANGE SCAN 거래_X02 (cr=140 pr=21 pw=0 time=326 us)

- ① 상품_X01 인덱스의 컬럼 순서를 조정한다.
- ② 상품_X01 인덱스에 컬럼을 추가한다.
- ③ 거래_X02 인덱스에 컬럼을 추가한다.
- ④ 거래_X02 대신 거래 X_01 인덱스를 사용한다.

[상품 테이블 조회 조건]

```
where p. 상품분류코드 = 'KTG'  
and p. 공급업체코드 = 'SP83732'  
and p. 상품가격 between 10000 and 100000
```

[사용된 인덱스] 상품_X01 : 상품분류코드 + 상품가격 + 공급업체코드
→ 인덱스의 공급업체코드가 후행에 위치해 상품가격 범위조회에 비효율 존재

[조인 조건]

```
and t. 상품코드 = p. 상품코드
```

[거래 테이블 조회 조건]

```
and t.거래일자 between '20210101' and '20210131'
```

[사용된 인덱스] 거래_X02 : 상품코드 + 거래구분코드 + 거래일자
→ 조인조건인 상품코드와 거래 테이블의 조회 조건인 거래일자 사이에
"거래구분코드"가 존재해 인덱스 효율적인 인덱스 스캔이 되지 않음.

① 상품_X01 인덱스의 컬럼 순서를 조정한다. (O)

② 상품_X01 인덱스에 컬럼을 추가한다. (X)

→ 상품 인덱스에 인덱스를 추가해도 결과집합을 줄이는데 큰 효과가 없다.

③ 거래_X02 인덱스에 컬럼을 추가한다. (X)

→ 거래_X02에 인덱스를 추가하는 것보다 상품코드 + 거래일자로 인덱스를 변경하는 것이 더 효율적이다.

④ 거래_X02 대신 거래_X01 인덱스를 사용한다. (X)

→ 조인 컬럼이 상품코드이기 때문에 거래_X01로 변경하면 거래일자를 먼저 탐색하는 비효율이 발생한다.

6. 다음 중 NL 조인의 특징으로 가장 부적절한 것은?

- ① 랜덤 액세스 위주의 조인 방식이다.
- ② 조인할 대상 레코드가 아무리 많아도 `ArraySize`에 해당하는 최초 N건을 빠르게 출력할 수 있다.
- ③ 인덱스 유무, 인덱스 구성에 의해 성능이 크게 달라진다.
- ④ 부분범위 처리가 가능한 상황에서만 효과적인 조인 방식이다.

- ① 랜덤 액세스 위주의 조인 방식이다.
- ② 조인할 대상 레코드가 아무리 많아도 ArraySize에 해당하는 최초 N건을 빠르게 출력할 수 있다.
- ③ 인덱스 유무, 인덱스 구성에 의해 성능이 크게 달라진다.
- ④ 부분범위 처리가 가능한 **상황에서만** 효과적인 조인 방식이다. (X)
→ 부분범위 처리가 가능하지 않은 상황에서도 효과적이다.

7. 다음 두 SQL을 위한 PRA_HST_STC 테이블의 최적 인덱스 구성안을 고르시오.

[SQL 1]

```
SELECT /*+ ORDERED USE_NL(B) */ *  
FROM PRA_HST_STC A, ODM_TRMS B  
WHERE A.SALE_ORG_ID = :saleOrgId  
AND   A.STRD_GRP_ID = B.STRD_GRP_ID  
AND   A.STRD_ID = B.STRD_ID  
ORDER BY A.STC_DT
```

[SQL 2]

```
SELECT /*+ ORDERED USE_NL(B) */ *  
FROM PRA_HST_STC A, ODM_TRMS B  
WHERE A.USER_ID = :userId  
AND   A.SALE_ORG_ID = :saleOrgId  
AND   A.STRD_GRP_ID = B.STRD_GRP_ID  
AND   A.STRD_ID = B.STRD_ID  
ORDER BY A.STC_DT
```

① PRA_HST_STC_X1 : SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT

PRA_HST_STC_X2 : USER_ID + SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT

② PRA_HST_STC.X1 : SALE_ORG_ID + USER_ID + STRD_GRP_ID + STRD_ID + STC_DT

③ PRA_HST_STC.X1 : SALE_ORG_ID + USER_ID + STC_DT + STRD_GRP_ID + STRD_ID

④ PRA_HST_STCX1: SALE_ORG_ID + USER_ID

조건절 분석

[SQL 1]

```
SELECT /*+ ORDERED USE_NL(B) */ *
FROM PRA_HST_STC A, ODM_TRMS B
WHERE A.SALE_ORG_ID = :saleOrgId
AND A.STRD_GRP_ID = B.STRD_GRP_ID
AND A.STRD_ID = B.STRD_ID
ORDER BY A.STC_DT
```

[SQL 2]

```
SELECT /*+ ORDERED USE_NL(B) */ *
FROM PRA_HST_STC A, ODM_TRMS B
WHERE A.USER_ID = :userId
AND A.SALE_ORG_ID = :saleOrgId
AND A.STRD_GRP_ID = B.STRD_GRP_ID
AND A.STRD_ID = B.STRD_ID
ORDER BY A.STC_DT
```

구분	설명
공통 사용	- A.STRD_GRP_ID = B.STRD_GRP_ID - A.STRD_ID = B.STRD_ID - A.SALE_ORG_ID = :saleOrgId
SQL2	- A.USER_ID = :userId
1) SALE_ORG_ID + USER_ID + STC_DT 2) SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT SALE_ORG_ID + USER_ID + STRD_GRP_ID + STRD_ID + STC_DT	

→ 위 두가지 형태가 나오지만, STRD_GRP_ID, STRD_ID는 조인을 위해 필요한 것이지 PRA_HST_STC 테이블 인덱스 구성에는 필요 없는 컬럼

- ① PRA_HST_STC_X1 : SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT
PRA_HST_STC_X2 : USER_ID + SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT
- ② PRA_HST_STC.X1 : SALE_ORG_ID + USER_ID + STRD_GRP_ID + STRD_ID + STC_DT
- ③ PRA_HST_STC.X1 : SALE_ORG_ID + USER_ID + STC_DT + STRD_GRP_ID + STRD_ID
- ④ PRA_HST_STCX1: SALE_ORG_ID + USER_ID (O)

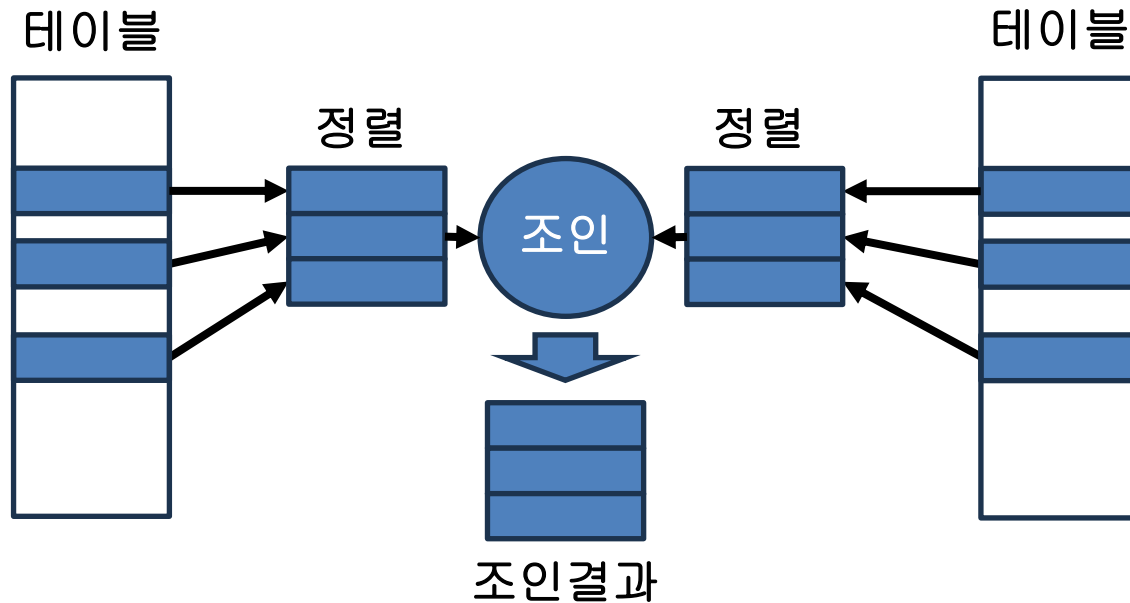
8. 주문 테이블을 기준으로 고객 테이블과 소트 머지 조인하도록 유도하고자 할 때 빈 칸 (가)에 들어갈 오라클 힌트는?

```
select /*+      (가)      */  
        o. 주문번호, o. 고객번호, c. 고객명, c. 전화번호, o. 주문금액  
From 주문 o, 고객 C  
where o.주문일자 >= trunc(sysdate)  
and   c.고객번호 = o.고객번호
```

- ① use_merge (o c)
- ② ordered merge (c)
- ③ ordered use_merge (c)
- ④ leading(o) sort_merge (c)

소트 머지 조인 (Sort Merge Join)의 개념

두 테이블을 먼저 각각 정렬을 한 다음에 집합을 Merge 하면서
조인하는 조인기법



- NL 조인의 랜덤 액세스와 다르게 Full Table Scan으로 조회
- 정렬할 데이터가 많아 메모리가 부족할 경우 디스크를 이용
- 비동등 조인(Non-equi join)도 조인이 가능

소트 머지 조인 (Sort Merge Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ ordered use_merge(A) */
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME
FROM DEPT B, EMP A
WHERE A.DEPTNO = B.DEPTNO;

select plan_table_output from
table(dbms_xplan.display('plan_table',null,'serial'));
    
```

- USE_MERGE, ordered 힌트를 통해서 실행계획 제어가 가능
DEPT 테이블을 DEPTNO 컬럼으로 정렬 및
EMP 테이블을 DEPTNO 컬럼으로 정렬 후 조인
- **USE_MERGE** : 소트 머지 방식 조인 사용
- **ordered** : FROM 절에 기술된 순서대로 조인

① `use_merge ( o c )` (X)

→ 테이블 순서를 지정하지 않음

② `ordered merge ( c )` (X)

→ 힌트 구문이 잘못되었음

③ `ordered use_merge ( c )` (O)

④ `leading(o) sort_merge ( c )` (X)

→ 힌트 구문이 잘못되었음

(정답) ③

9. 아래 실행계획이 나타나도록 옵티마이저 힌트 중 가장 적절한 것은?

```
select d. deptno, d.dname, e.empno, e.ename
from emp e, dept d
where d. deptno = e. deptno
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		14	336	8 (25)	00:00:01
1	MERGE JOIN		14	336	8 (25)	00:00:01
2	SORT JOIN		4	44	4 (25)	00:00:01
3	TABLE ACCESS FULL	DEPT	4	44	3 (0)	00:00:01
4	SORT JOIN		14	182	4 (25)	00:00:01
5	TABLE ACCESS FULL	EMP	14	182	3(0)	00:00:01

- ① leading(d) use_merge(e)
- ② ordered sort_merge(e)
- ③ leading(e) use_merge(d)
- ④ ordered sort_merge(d)

① leading(d) use_merge(e) (O)

② ordered sort_merge(e) (X)

→ 힌트 구문이 잘못되었음

③ leading(e) use_merge(d) (X)

→ 테이블 액세스 순서가 잘못 되었음

④ ordered sort_merge(d) (X)

→ 힌트 구문이 잘못되었음

(정답) ①

10. 다음 중 소트 머지 조인의 특징으로 가장 부적절한 것은?

- ① 랜덤 액세스가 아닌 스캔 위주의 조인 방식이다.
- ② 양쪽 집합을 개별적으로 읽어서 정렬하므로 조인 컬럼에 인덱스가 없어도 상관없다.
- ③ 양쪽 테이블 모두 조인 컬럼에 인덱스가 있으면 따로 정렬을 수행하지 않아도 되므로 매우 유리하다.
- ④ 두 테이블을 각각 읽어 조인 대상 집합을 줄일 수 있을 때 매우 효과적이다.

소트 머지 조인 (Sort Merge Join)의 특징

특징	설명
양쪽 정렬	- 조인하기 전에 양쪽 테이블(집합)을 조인키로 정렬을 한 후에 조인
두 테이블의 건수에 따라 일량 결정	- 양쪽 테이블(집합)을 각각 정렬한 후 조인을 하므로, 각 집합의 크기에 따라 일량 결정 - 테이블별 검색 조건에 의해 전체 일량이 달라짐
스캔방식 조인	- NL 조인은 랜덤 액세스 방식의 조인이라면 소트 머지 조인은 스캔 위주 조인 방식 - Inner 테이블을 반복 액세스 하지 않아 머지 과정에서 랜덤 액세스가 발생하지 않음 - 하지만, 각 테이블의 대상집합을 찾을 때 인덱스를 이용한 랜덤 액세스가 처리될 수 있고 이때 랜덤 액세스량이 많다면 소트 머지 조인의 이점이 사라짐

- 위 특징을 종합할 때 소트 머지 조인은 대량 데이터를 조회할 때 적합한 조인기법 이다. (배치에서 사용됨)
- 비동등 조인(Non-equi join)의 대량 데이터 조인
- 카테시안 곱
- **랜덤 액세스** : 한번에 하나의 블록만 액세스하는 방식

- ① 랜덤 액세스가 아닌 스캔 위주의 조인 방식이다.
- ② 양쪽 집합을 개별적으로 읽어서 정렬하므로 조인 컬럼에 인덱스가 없어도 상관없다.
- ③ 양쪽 테이블 모두 조인 컬럼에 인덱스가 있으면 따로 정렬을 수행하지 않아도 되므로 매우 유리하다. (X)
→ 인덱스를 사용하지 않고 양쪽 테이블의 조인키로 PGA영역에 정렬 후 조인
다만, 첫번째 테이블은 조인키가 인덱스로 있으면 PGA영역에 정렬을 하지 않음
- ④ 두 테이블을 각각 읽어 조인 대상 집합을 줄일 수 있을 때 매우 효과적이다.

11. 해시 조인과 비교할 때 소트 머지 조인의 장점을 고르시오.

- ① 메모리를 적게 사용한다.
- ② Temp 테이블스페이스를 적게 사용한다.
- ③ 조인 조건이 '='이 아닐 때도 사용할 수 있다.
- ④ 양쪽 집합이 모두 대용량일 때 해시 조인보다 빠르다.

① 메모리를 적게 사용한다. (X)

→ 양쪽 테이블 집합을 PGA영역에서 정렬하기 때문에 Hash Join보다 메모리를 더 사용한다.

② Temp 테이블스페이스를 적게 사용한다. (X)

→ 상황에 따라 다르겠지만 메모리를 Sort Merge Join이 더 사용하는 만큼 Temp 테이블스페이스도 더 많이 사용할 수 있다.

③ 조인 조건이 '='이 아닐 때도 사용할 수 있다. (O)

④ 양쪽 집합이 모두 대용량일 때 해시 조인보다 빠르다. (X)

→ 조인 조건이 등치(=)인 경우 일반적으로 해시조인이 더 빠르다.

(해시조인은 등치(=)인 경우만 작동하기 때문에 등치가 아닌 경우에만 소트머지조인 사용)

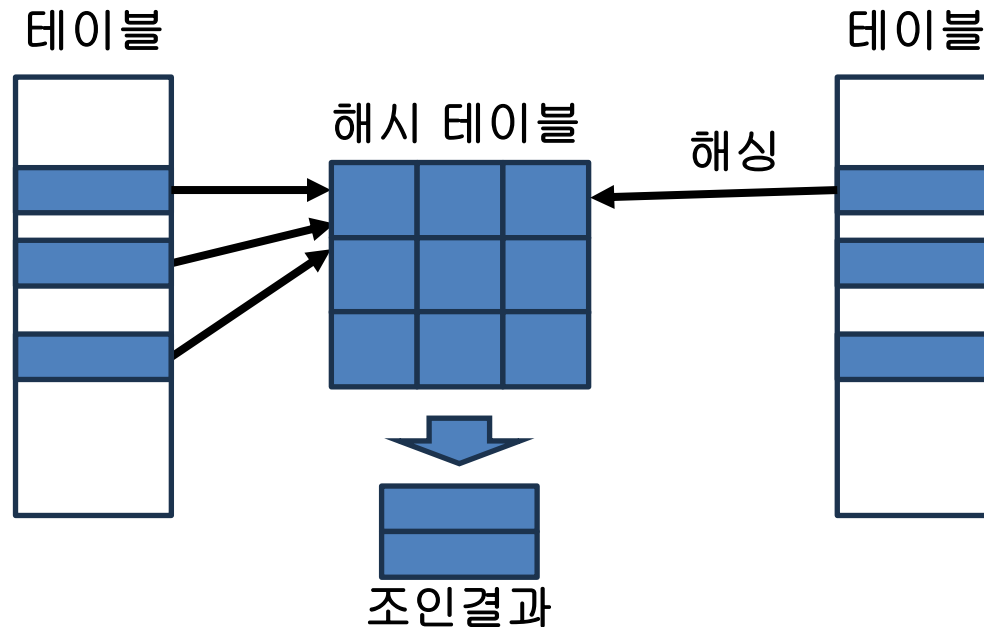
12. 다음 중 Hash Join에 대한 설명으로 가장 부적절한 것을 2개 고르시오.

- ① 조인 연결고리에 equi-join 조건이 하나라도 있어야 한다.
- ② 일반적으로 큰 집합으로 해시 테이블을 생성하고서 작은 집합을 읽으면서 이 해시 테이블을 탐색하는게 유리하다.
- ③ OLTP 환경에서 수행빈도가 아주 높은 쿼리라도 부담 없이 사용할 수 있다.
- ④ Build Input으로 선택된 집합의 조인 칼럼에는 중복 값이 거의 없어야 효과적이다

.

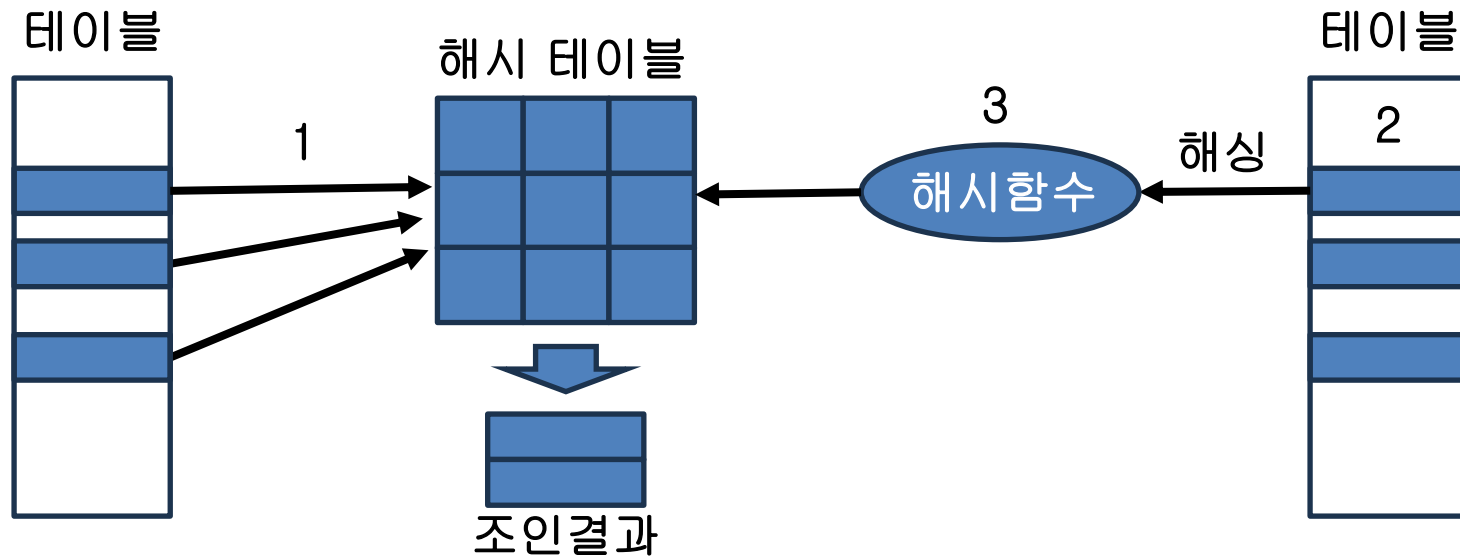
해시 조인 (Hash Join)의 개념

두 테이블 중 작은 테이블을 해시 영역(Hash Area)에 해시 테이블을 생성하고 큰 집합을 읽어 해시 테이블을 탐색해 조인



- NL Join의 랜덤 액세스의 문제와 소트 머지 조인의 정렬 문제점을 해결하는 대안으로 등장
- 동등 조인(Equi join)만 조인이 가능

해시 조인 (Hash Join)의 조인과정



NO	설명
1	두 테이블 중 작은 테이블에서 조인 조건에 해당되는 행의 데이터로 해시 테이블을 생성(Build Input)
2	조인할 테이블에서 조인 조건에 해당하는 행을 탐색
3	조인할 테이블의 조인 컬럼 기준으로 해시함수를 적용해 해시테이블에서 적합한 버킷을 탐색
4	조인 성공 시 추출 버퍼에 저장
5	2~4번 반복

- ① 조인 연결고리에 equi-join 조건이 하나라도 있어야 한다. (O)
- ② 일반적으로 큰 집합으로 해시 테이블을 생성하고서 작은 집합을 읽으면서 이 해시 테이블을 탐색하는게 유리하다. (X)
→ 작은 테이블을 해시 테이블로 생성하는게 유리
- ③ OLTP 환경에서 수행빈도가 아주 높은 SQL이라도 부담 없이 사용할 수 있다. (X)
→ 수행빈도가 높은 SQL을 HASH 조인을 할 경우 CPU Usage가 높아질 수 있다.
(해시테이블을 만들어야 해서)
- ④ Build Input으로 선택된 집합의 조인 칼럼에는 중복 값이 거의 없어야 효과적이다. (O)

(정답) ②, ③

13. 다음의 실행계획이, 나타나도록 유도하고자 할 때, 빈칸 (가)에 들어한 오라클 힌트로 가장 부적합한 것은?

[실행계획]

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		14	364	7 (15)	00:00:01
1	HASH JOIN		14	364	7 (15)	00:00:01
2	TABLE ACCESS FULL	DEPT	4	52	3 (0)	00:00:01
3	TABLE ACCESS FULL	EMP	14	182	7 (15)	00:00:01

[SQL]

```
select /*+ (가) */ d.deptno, d.dname, e.empno, e.ename
from emp e, dept d
where d.deptno = e. deptno
```

- ① leading(d) use_hash(e)
- ② ordered use_hash(e)
- ③ ordered use_hash(d) swap_join_inputs (d)
- ④ use_hash(e d) swap_join_inputs(d)

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
SELECT /*+ ordered use_hash(A) */ A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME
FROM DEPT B, EMP A
WHERE A.DEPTNO = B.DEPTNO;

select plan_table_output from table(dbms_xplan.display(null,null, 'serial'));
```

- USE_HASH, ordered 힌트를 통해서 실행계획 제어가 가능
DEPT 테이블과 EMP 테이블 중 작은 테이블을 build input으로
선택 후 해시테이블을 생성 후 조인
- **USE_HASH** : 해시 방식 조인 사용
- **ordered** : FROM 절에 기술된 순서대로 조인

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
SELECT /*+ ordered use_hash(A) swap_join_inputs(A) */
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME
FROM DEPT B, EMP A
WHERE A.DEPTNO = B.DEPTNO;

select plan_table_output from table(dbms_xplan.display(null,null, 'serial'));
```

– SWAP_JOIN_INPUTS : build input 테이블을 선택

```
SELECT /*+ leading(B, A, C) use_hash(A) use_hash(c) */
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM DEPT B, EMP A, EMP C
WHERE A.DEPTNO = B.DEPTNO
AND A.MGR = C.EMPNO;
```

- 3개 이상 테이블을 조인하면 leading 힌트로 조인순서를 지정
- Leading 힌트의 첫번째 파라미터 테이블은 build input 선택

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(B) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
  
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	280	6 (0)	00:00:01
* 1	HASH JOIN		7	280	6 (0)	00:00:01
* 2	HASH JOIN		7	210	4 (0)	00:00:01
3	TABLE ACCESS FULL DEPT		4	52	2 (0)	00:00:01
4	TABLE ACCESS FULL EMP		7	119	2 (0)	00:00:01
5	TABLE ACCESS FULL EMP		7	70	2 (0)	00:00:01

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(C) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM DEPT B, EMP A, EMP C
WHERE A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
  
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		7	280	6	(0)	00:00:01
* 1	HASH JOIN		7	280	6	(0)	00:00:01
2	TABLE ACCESS FULL	EMP	7	70	2	(0)	00:00:01
* 3	HASH JOIN		7	210	4	(0)	00:00:01
4	TABLE ACCESS FULL	DEPT	4	52	2	(0)	00:00:01
5	TABLE ACCESS FULL	EMP	7	119	2	(0)	00:00:01

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) no_swap_join_inputs(C) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
  
```

– build input 선택하지 않을 경우 no_swap_join_inputs 힌트 사용

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	280	6 (0)	00:00:01
* 1	HASH JOIN		7	280	6 (0)	00:00:01
* 2	HASH JOIN		7	210	4 (0)	00:00:01
3	TABLE ACCESS FULL DEPT		4	52	2 (0)	00:00:01
4	TABLE ACCESS FULL EMP		7	119	2 (0)	00:00:01
5	TABLE ACCESS FULL EMP		7	70	2 (0)	00:00:01

① leading(d) use_hash(e) (O)

→ leading 힌트로 액세스 순서를 DEPT 테이블로 지정하였고 조인방법은 use_hash로 선택

② ordered use_hash(e) (X)

→ DEPT 테이블을 먼저 읽어야 하기 때문에 ordered 는 적합하지 않다.

③ ordered use_hash(d) swap_join_inputs(d) (O)

→ build Input 테이블을 d로 지정 했기 때문에 DEPT 테이블을 먼저 읽는다. 조인방법은 해시

④ use_hash(e d) swap_join_inputs(d) (O)

→ build Input 테이블을 d로 지정 했기 때문에 DEPT 테이블을 먼저 읽는다. 조인방법은 해시

14. 다음 중 조인 순서에 가장 영향을 적게 받는 조인 메소드를 고르시오.

- ① NL 조인
- ② 소트 머지 조인
- ③ 해시 조인
- ④ 해시 아우터 조인

① NL 조인 (X)

→ 드라이빙 테이블의 스캔 범위에 따라 전체 스캔량이 정해지므로 조인순서의 영향이 크다.

② 소트 머지 조인 (O)

→ 소트 머지 조인은 두 테이블을 정렬을 한 후 조인하기 때문에 조인순서의 영향이 크지 않다.

③ 해시 조인 (X)

→ 작은 크기의 테이블을 build input 해야지 유리하므로 조인순서에 영향이 크다.

④ 해시 아우터 조인 (X)

→ 작은 크기의 테이블을 build input 해야지 유리하므로 조인순서에 영향이 크다.

15. 아래 SQL에 대한 튜닝 방안으로 가장 부적절한 것은?

[데이터]

대리점 : 1,000개

상품판매실적 : 월평균 100만 건

[인덱스 구성]

대리점_PK : 대리점코드

상품판매실적_PX : 대리점코드 + 상품코드 + 판매일자

상품판매실적_X1 : 판매일자 + 상품코드

```
SELECT A. 대리점명, SUM(B. 매출금액) 매출금액
FROM 대리점 A, 상품판매실적 B
WHERE A. 대리점코드 = B. 대리점코드
AND B. 상품코드 IN ( 'A1847', 'Z04131' )
AND B. 판매일자 BETWEEN ' 20210101' AND '20210331'
GROUP BY B. 대리점코드, A. 대리점명
ORDER BY 1, 2
```

Rows	Row Source Operation
-----	-----
30	SORT GROUP BY
90000	NESTED LOOPS
90000	TABLE ACCESS BY INDEX ROWID 상품판매실적
90000	INDEX RANGE SCAN 상품판매실적_X1 (NONUNIQUE)
90000	TABLE ACCESS BY INDEX ROWID 대리점
90000	INDEX UNIQUE SCAN 대리점_PK (UNIQUE)

- ① 상품판매실적_X1 인덱스를 「 상품코드 + 판매일자 」 순으로 변경한다.
- ② 조인 순서를 변경한다.
- ③ 판매 일자 조건을 만족하는 상품판매실적을 대리점코드로 GROUP BY 한 후 대리점 테이블과 조인하도록 SQL을 변환한다.
- ④ 해시 조인으로 유도한다.

① 상품판매실적_X1 인덱스를 「 상품코드 + 판매일자 」 순으로 변경한다.

판매일자가 BETWEEN 이기 때문에
상품코드는 필터조건으로 사용됨
→ 순서를 변경하면 성능개선에
도움이 됨

```
WHERE A. 대리점코드 = B. 대리점코드
AND B. 상품코드 IN ( 'A1847', 'Z04131' )
AND B. 판매일자 BETWEEN ' 20210101' AND '20210331'
GROUP BY B. 대리점코드, A. 대리점명
```

② 조인 순서를 변경한다.

전체 대리점에 대한 'A1847', 'Z04131' 상품에 대한 판매 내역을 조회하게 된다.

Rows	Row Source	Operation
30	SORT	GROUP BY

하지만 1,000개의 대리점에서 두상품을 판매한 대리점은 30개

→ 상품 판매를 하지 않은 970개의 대리점의 상품판매실적을 조회해야 함

그리고 상품코드가 IN 조건이기 때문에 상품판매실적_PX 인덱스를 두 번 스캔함.

- ③ 판매 일자 조건을 만족하는 상품판매실적을 대리점코드로 GROUP BY 한 후 대리점 테이블과 조인하도록 SQL을 변환한다.

GROUP BY를 한 뒤에
대리점코드와 조인하면
대리점 테이블 액세스
건수가 줄어든다.

단, NO_MERGE 힌트로
뷰머지를 방지해야 한다.

Rows	Row Source Operation
30	SORT GROUP BY
90000	NESTED LOOPS
90000	TABLE ACCESS BY INDEX ROWID 상품판매실적
90000	INDEX RANGE SCAN 상품판매실적_X1 (NONUNIQUE)
90000	TABLE ACCESS BY INDEX ROWID 대리점
90000	INDEX UNIQUE SCAN 대리점_PK (UNIQUE)

- ④ 해시 조인으로 유도한다.

해시조인을 하게되면 1,000건의 로우를 가지는 대리점 테이블을 build input으로
사용해 PGA 영역에 해시맵을 만들기 때문에 SGA 버퍼 캐시를 반복적으로
스캔하는걸 줄일 수 있다.

- ① 상품판매실적_X1 인덱스를 「 상품코드 + 판매일자 」 순으로 변경한다. (O)
- ② 조인 순서를 변경한다. (X)
- ③ 판매 일자 조건을 만족하는 상품판매실적을 대리점코드로 GROUP BY 한 후 대리점 테이블과 조인하도록 SQL을 변환한다. (O)
- ④ 해시 조인으로 유도한다. (O)

16. 아래 SQL에 대한 튜닝 방안으로 가장 적절한 것을 고르시오

[데이터]

상품 : 50만 rows

상품상세 : 500만 rows

상품변경이력 : 1,000만 rows

[인덱스 구성]

상품_PK : 상품코드

상품상세_PK : 상품코드 + 상품상세코드

상품상세_X1 : 등록일시 + 상품코드

상품변경이력_PK : 상품코드 + 변경일시

```
create table 상품상세_temp
```

```
nologging
```

```
as
```

```
select /*+ ordered use_hash(b) use_hash(c) swap_join_inputs(c) */ *
```

```
from 상품상세 a, 상품변경이력 b, 상품 C
```

```
where b. 상품코드 = a. 상품코드
```

```
and b. 변경일시 like substr(a.등록일시, 1, 8) '%'
```

```
and c. 상품코드 = a. 상품코드
```

- ① `use_hash(b)` 힌트를 `use_nl(b)`로 수정한다.
- ② `ordered use_hash(b)` 힌트를 `leading(b) use_hash(a)`로 수정한다.
- ③ 힌트 전체를 `leading(c a b) use_hash(a) use_hash(b) swap_join_inputs(a)`로 수정한다.
- ④ 두 번째 조건절을 아래와 같이 수정한다.
`and substr(b.변경일시, 1, 8) = substr(a.등록일시, 1, 8)`

① `use_hash(b)` 힌트를 `use_nl(b)`로 수정한다.

해시 조인을 NL 조인으로 변경하면, 대량의 데이터에서 성능이 떨어질 수 있음
→ LIKE 조건절을 등치(=) 변경하는 것이 훨씬 효과적임

② `ordered use_hash(b)` 힌트를 `leading(b) use_hash(a)`로 수정한다.

`build input`이 상품(C) 테이블이어서 먼저 액세스를 하게 되는데
50만 rows와 1,000만 rows를 조인을 하게 되기 때문에 비효율이 존재한다.

③ 힌트 전체를 `leading(c a b) use_hash(a) use_hash(b) swap_join_inputs(a)`로 수정한다.

`build input` 테이블이 500만 rows라서 비효율적이다.

④ 두 번째 조건절을 아래와 같이 수정한다.

`and substr(b.변경일시, 1, 8) = substr(a.등록일시, 1, 8)`

조인 조건을 Like 에서 등치(=)로 변경되면 해시맵을 구성할 수 있게 되어
해시조인이 가능

- ① use_hash(b) 힌트를 use_nl(b)로 수정한다. (X)
- ② ordered use_hash(b) 힌트를 leading(b) use_hash(a)로 수정한다. (X)
- ③ 힌트 전체를 leading(c a b) use_hash(a) use_hash(b) swap_join_inputs(a)로 수정한다. (X)
- ④ 두 번째 조건절을 아래와 같이 수정한다.
and substr(b.변경일시, 1, 8) = substr(a.등록일시, 1, 8) (O)

17. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and   c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS
1  0    HASH JOIN
2  1      TABLE ACCESS (FULL) OF ' 제품' (TABLE)
3  1      HASH JOIN
4  3          TABLE ACCESS (FULL) OF '구매' (TABLE)
5  3          TABLE ACCESS (FULL) OF '상품' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(C) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
    
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		7	280	6	(0)	00:00:01
* 1	HASH JOIN		7	280	6	(0)	00:00:01
2	TABLE ACCESS FULL	EMP	7	70	2	(0)	00:00:01
* 3	HASH JOIN		7	210	4	(0)	00:00:01
4	TABLE ACCESS FULL	DEPT	4	52	2	(0)	00:00:01
5	TABLE ACCESS FULL	EMP	7	119	2	(0)	00:00:01

① `/*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */`

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time	

0	SELECT STATEMENT		1	61	6 (0)	00:00:01	
* 1	HASH JOIN		1	61	6 (0)	00:00:01	
* 2	HASH JOIN		1	41	4 (0)	00:00:01	
3	TABLE ACCESS FULL	제품	1	14	2 (0)	00:00:01	
4	TABLE ACCESS FULL	상품	1	27	2 (0)	00:00:01	
5	TABLE ACCESS FULL	구매	1	20	2 (0)	00:00:01	

② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time	

0	SELECT STATEMENT		1	61	6 (0)	00:00:01	
* 1	HASH JOIN		1	61	6 (0)	00:00:01	
2	TABLE ACCESS FULL	구매	1	20	2 (0)	00:00:01	
* 3	HASH JOIN		1	41	4 (0)	00:00:01	
4	TABLE ACCESS FULL	제품	1	14	2 (0)	00:00:01	
5	TABLE ACCESS FULL	상품	1	27	2 (0)	00:00:01	

③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time	
0	SELECT STATEMENT		1	61	6 (0)	00:00:01	
* 1	HASH JOIN		1	61	6 (0)	00:00:01	
* 2	HASH JOIN		1	47	4 (0)	00:00:01	
3	TABLE ACCESS FULL	구매	1	20	2 (0)	00:00:01	
4	TABLE ACCESS FULL	상품	1	27	2 (0)	00:00:01	
5	TABLE ACCESS FULL	제품	1	14	2 (0)	00:00:01	

④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time	

0	SELECT STATEMENT		1	61	6 (0)	00:00:01	
* 1	HASH JOIN		1	61	6 (0)	00:00:01	
2	TABLE ACCESS FULL	제품	1	14	2 (0)	00:00:01	
* 3	HASH JOIN		1	47	4 (0)	00:00:01	
4	TABLE ACCESS FULL	구매	1	20	2 (0)	00:00:01	
5	TABLE ACCESS FULL	상품	1	27	2 (0)	00:00:01	

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */ (O)

18. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and   c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS
1  0      HASH JOIN
2  1          HASH JOIN
3  2          TABLE ACCESS (FULL) OF '구매' (TABLE)
4  2          TABLE ACCESS (FULL) OF '상품' (TABLE)
5  2          TABLE ACCESS (FULL) OF '제품' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(B) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
  
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	280	6 (0)	00:00:01
* 1	HASH JOIN		7	280	6 (0)	00:00:01
* 2	HASH JOIN		7	210	4 (0)	00:00:01
3	TABLE ACCESS FULL DEPT		4	52	2 (0)	00:00:01
4	TABLE ACCESS FULL EMP		7	119	2 (0)	00:00:01
5	TABLE ACCESS FULL EMP		7	70	2 (0)	00:00:01

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */ (O)
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

19. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and   c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS
1  0    HASH JOIN
2  1      TABLE ACCESS (FULL) OF ' 구매' (TABLE)
3  1      HASH JOIN
4  3          TABLE ACCESS (FULL) OF '제품' (TABLE)
5  3          TABLE ACCESS (FULL) OF '상품' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(C) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM DEPT B, EMP A, EMP C
WHERE A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
  
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		7	280	6	(0)	00:00:01
* 1	HASH JOIN		7	280	6	(0)	00:00:01
2	TABLE ACCESS FULL	EMP	7	70	2	(0)	00:00:01
* 3	HASH JOIN		7	210	4	(0)	00:00:01
4	TABLE ACCESS FULL	DEPT	4	52	2	(0)	00:00:01
5	TABLE ACCESS FULL	EMP	7	119	2	(0)	00:00:01

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */ (O)
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

(정답) ②

20. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and   c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS
1  0      HASH JOIN
2  1          HASH JOIN
3  2          TABLE ACCESS (FULL) OF '제품' (TABLE)
4  2          TABLE ACCESS (FULL) OF '상품' (TABLE)
5  2          TABLE ACCESS (FULL) OF '구매' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

해시 조인 (Hash Join)의 실행계획 제어

```

explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(B) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
  
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	280	6 (0)	00:00:01
* 1	HASH JOIN		7	280	6 (0)	00:00:01
* 2	HASH JOIN		7	210	4 (0)	00:00:01
3	TABLE ACCESS FULL DEPT		4	52	2 (0)	00:00:01
4	TABLE ACCESS FULL EMP		7	119	2 (0)	00:00:01
5	TABLE ACCESS FULL EMP		7	70	2 (0)	00:00:01

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */ (O)
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

21. 오라클 11g 이상 버전에서 아래 SQL에 대해 나타날 수 있는 실행계획으로 가장 부적절한 것은?

```
select /*+ ordered use_nl(s) index(p) index(s) */ *
from 제품 p, 판매 s
where p.공급코드 = 'P20211212'
and s.제품코드 = p. 제품코드
and s.판매일자 between ' 20210101' and '20210131'
```

① Execution Plan

```
0  SELECT STATEMENT
1  0  NESTED LOOPS
2  1    TABLE ACCESS BY INDEX ROWID OF 제품
3  2      INDEX RANGE SCAN OF 제품_X1
4  1    TABLE ACCESS BY INDEX ROWID OF 판매
5  4      INDEX RANGE SCAN OF 판매_X1
```

② Execution Plan

```
0  SELECT STATEMENT
1  0  TABLE ACCESS BY INDEX ROWID OF 판매
2  1    NESTED LOOPS
3  2      TABLE ACCESS BY INDEX ROWID OF 제품
4  3          INDEX RANGE SCAN OF 제품_X1
5  2          INDEX RANGE SCAN OF 판매_X1
```

③ Execution Plan

```
0  SELECT STATEMENT
1 0  TABLE ACCESS BY INDEX ROWID OF 제품
2 1    NESTED LOOPS
3 2      INDEX RANGE SCAN OF 제품_X1
4 2      TABLE ACCESS BY INDEX ROWID OF 판매
5 4        INDEX RANGE SCAN OF 판매_X1
```

④ Execution Plan

```
0  SELECT STATEMENT
1 0  NESTED LOOPS
2 1    NESTED LOOPS
3 2      TABLE ACCESS BY INDEX ROWID OF 제품
4 3        INDEX RANGE SCAN OF 제품_X1
5 2        INDEX RANGE SCAN OF 판매_X1
6 1      TABLE ACCESS BY INDEX ROWID OF 판매
```

① Execution Plan

```
0  SELECT STATEMENT
1  0  NESTED LOOPS
2  1    TABLE ACCESS BY INDEX ROWID OF 제품
3  2      INDEX RANGE SCAN OF 제품_X1
4  1    TABLE ACCESS BY INDEX ROWID OF 판매
5  4      INDEX RANGE SCAN OF 판매_X1
```

/+ ordered use_nl(s) index(p) index(s) */ **

→ *index(p)*로 제품_X1, *index(s)*로 판매_X1 스캔 후 NL 조인
기본적인 NL 조인의 실행계획

② Execution Plan

```
0  SELECT STATEMENT
1 0  TABLE ACCESS BY INDEX ROWID OF 판매
2 1    NESTED LOOPS
3 2      TABLE ACCESS BY INDEX ROWID OF 제품
4 3        INDEX RANGE SCAN OF 제품_X1
5 2          INDEX RANGE SCAN OF 판매_X1
```

/+ ordered use_nl(s) index(p) index(s) */ **

→ 드라이빙 테이블(Outer 테이블)을 액세스 한 후
내부 테이블(Inner 테이블)을 액세스 하므로 가능
Prefetch 기능이 동작하는 실행계획

③ Execution Plan

```
0  SELECT STATEMENT
1 0  TABLE ACCESS BY INDEX ROWID OF 제품
2 1    NESTED LOOPS
3 2      INDEX RANGE SCAN OF 제품_X1
4 2      TABLE ACCESS BY INDEX ROWID OF 판매
5 4        INDEX RANGE SCAN OF 판매_X1
```

/+ ordered use_nl(s) index(p) index(s) */ **

→ 제품을 먼저 스캔하고 판매를 스캔해야 하지만 판매부터 스캔.

위와 같은 실행 계획은 나타날 수 없음

④ Execution Plan

```
0  SELECT STATEMENT
1 0  NESTED LOOPS
2 1   NESTED LOOPS
3 2    TABLE ACCESS BY INDEX ROWID OF 제품
4 3     INDEX RANGE SCAN OF 제품_X1
5 2     INDEX RANGE SCAN OF 판매_X1
6 1    TABLE ACCESS BY INDEX ROWID OF 판매
```

/+ ordered use_nl(s) index(p) index(s) */ **

→ 배치 I/O 실행계획

22. 아래 SQL을 위한 TBL_RPA_INFO 테이블의 인덱스 구성에 대한 설명으로 올바른 것은?

```
[ SQL ]
SELECT /*+ ORDERED USE_L(B) */ *
  FROM TBL_RPA_HST A, TBL_RPA_INFO B
 WHERE A.PRA_HIST_ID = :praHistId
    AND B.RPA_ID = A.RPA_ID
    AND B.RPA_GRP_ID = :rpaGrpId
    AND B.PROC_DT BETWEEN :procDt1 AND :procDt2
 ORDER BY A.RPA_HIST_DT
```

```
[ 인덱스 구성 ]
RPA_INFO_IDX_01 : RPA_GRP_ID + PROC_DT + RPA_ID
RPA_INFO_IDX_02 : RPA_ID + RPA_GRP_ID + PROC_DT
RPA_INFO_IDX_03 : RPA_GRP_ID + RPA_ID + PROC_DT
```

- ① 상수 또는 변수 조건을 앞에 두는 것이 유리하므로 RPA_INFO_IDX_01 인덱스가 가장 효과적이다.
- ② 조인 조건을 앞에 두는 것이 유리하므로 다른 인덱스보다 RPA_INFO_IDX_02가 효과적이다.
- ③ RPA_INFO_IDX_02, RPA_INFO_IDX_03 인덱스가 최적이며, 둘 간에 성능 차이는 없다.
- ④ RPA_INFO_IDX_01, RPA_INFO_IDX_02, RPA_INFO_IDX_03 인덱스 모두 최적이다.

```
WHERE A.PRA_HIST_ID = :praHistId 일반조건
      AND B.RPA_ID = A.RPA_ID 조인조건
      AND B.RPA_GRP_ID = :rpaGrpId 일반조건
      AND B.PROC_DT BETWEEN :procDt1 AND :procDt2 일반조건
ORDER BY A.RPA_HIST_DT
```

[인덱스 구성]

RPA_INFO_IDX_01 : RPA_GRP_ID + PROC_DT + RPA_ID

→ 인덱스 조건인 RPA_ID 가 후미 컬럼으로 있어서 JOIN 효율이 떨어짐

RPA_INFO_IDX_02 : RPA_ID + RPA_GRP_ID + PROC_DT

→ 조인조건, 일반조건 둘 다 만족

RPA_INFO_IDX_03 : RPA_GRP_ID + RPA_ID + PROC_DT

→ RPA_GRP_ID가 바인드 변수로 JOIN을 고려하면 등치(=)조건으로 입력되어 있기 때문에 이 SQL에서 효율적으로 동작함

① 상수 또는 변수 조건을 앞에 두는 것이 유리하므로 RPA_INFO_IDX_01 인덱스가 가장 효과적이다. (X)

→ 조인 컬럼이 후미에 있어 많은 I/O가 발생됨

② 조인 조건을 앞에 두는 것이 유리하므로 다른 인덱스보다 RPA_INFO_IDX_02가 효과적이다. (X)

→ 현재 SQL 기준으로 RPA_INFO_IDX_02나 RPA_INFO_IDX_03이나 비슷함

③ RPA_INFO_IDX_02, RPA_INFO_IDX_03 인덱스가 최적이며, 둘 간에 성능 차이는 없다.

(O)

④ RPA_INFO_IDX_01 , RPA_INFO_IDX_02, RPA_INFO_IDX_03 인덱스 모두 최적이다. (X)

→ RPA_INFO_IDX_01 비효율적임

23. FROM 절에 나열한 주문, 고객, 결제방식 순으로 조인하되, 주문과 고객 테이블은 NL 조인, 결제방식과는 해시 조인하도록 적절한 힌트는?

```
select *  
from 주문 o, 고객 c, 결제방식 t  
where o.주문일자 >= trunc(sysdate)  
and c.고객번호 = 0.고객번호  
and t.결제방식코드 = 0.결제방식코드
```

- ① /*+ ordered use_nl(c) use_hash(t) */
- ② /*+ ordered use_nl(t) use_hash(c) */
- ③ /*+ leading(o c t) use_nl(t) use_hash(c) */
- ④ /*+ leading(c t o) use_nl(c) use_hash(t) */

① /*+ ordered use_nl(c) use_hash(t) */ (O)

② /*+ ordered use_nl(t) use_hash(c) */ (X)

→ 고객을 hash 조인, 결제방식을 nl 조인 사용

③ /*+ leading(o c t) use_nl(t) use_hash(c) */ (X)

→ 고객을 hash 조인, 결제방식을 nl 조인 사용

④ /*+ leading(c t o) use_nl(c) use_hash(t) */ (X)

→ 테이블 액세스 순서가 잘못됨

24. 아래 SQL을 다음과 같은 실행계획이 생성되도록 옵티마이저 힌트를 기술하시오.

[인덱스 구성]

계약_PK : 계약번호

계약_X1 : 계약일자

가입상품_PK : 계약번호 + 상품번호

가입상품_X1 : 가입일자

가입상품정보_PK : 계약번호 + 상품코드 + 상품정보코드

상품_PK : 상품코드

```
select a.계약번호, a.계약명, b.상품코드, b.가입일자, b.할인률, c.상품정보코드, d.상품명
  from 계약 a, 가입상품 b, 가입상품정보 c, 상품 d
 where a.계약일자 = :ctntDt
       and b.계약번호 = a.계약번호
       and b.가입일자 = :joinDt
       and c.계약번호 = b.계약번호
       and c.상품코드 = b.상품코드
       and c.상품정보코드 like 'A%'
       and d.상품코드 = c.상품정보코드
```

24. 아래 SQL을 다음과 같은 실행계획이 생성되도록 옵티마이저 힌트를 기술하시오.

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS
1      0      HASH JOIN
2      1      TABLE ACCESS (BY INDEX ROWID) OF '상품 ' (TABLE)
3      2      INDEX (RANGE SCAN) OF '상품_PK' (INDEX (UNIQUE))
4      1      NESTED LOOPS
5      4      HASH JOIN
6      5      TABLE ACCESS (BY INDEX ROWID) OF '가입상품' (TABLE)
7      6      INDEX (RANGE SCAN) OF '가입상품_X1' (INDEX)
8      5      TABLE ACCESS (BY INDEX ROWID) OF '계약' (TABLE)
9      8      INDEX (RANGE SCAN) OF '계약_X1' (INDEX)
10     4      INDEX (RANGE SCAN) OF '가입부가상품_PK' (INDEX (UNIQUE))
```

[실행계획 분석]

Execution Plan

```

0      SELECT STATEMENT Optimizer=ALL_ROWS
1      0      HASH JOIN
2      1      TABLE ACCESS (BY INDEX ROWID) OF '상품 ' (TABLE)
3      2      INDEX (RANGE SCAN) OF '상품_PK' (INDEX (UNIQUE))
4      1      NESTED LOOPS
5      4      HASH JOIN
6      5      TABLE ACCESS (BY INDEX ROWID) OF '가입상품' (TABLE)
7      6      INDEX (RANGE SCAN) OF '가입상품_X1' (INDEX)
8      5      TABLE ACCESS (BY INDEX ROWID) OF '계약' (TABLE)
9      8      INDEX (RANGE SCAN) OF '계약_X1' (INDEX)
10     4      INDEX (RANGE SCAN) OF '가입상품정보_PK' (INDEX (UNIQUE))
  
```

- 1) 가입상품(가입상품_X1) -> 계약(계약_x1) ->
가입부가상품(가입상품정보_PK) -> 상품(상품PK) 순서대로 조인
- 2) 가입상품과 계약은 Hash Join
- 3) 가입부가상품은 NL 조인
- 4) 상품은 Hash 조인, 그리고 맨 위에 위치하는 걸로 보아 build input 으로 보임

[leading 힌트 사용시]

```
/*+ leading(b a c d) use_hash(a) use_nl(c) use_hash(d) swap_join_inputs(d)  
Index(b 가입상품_X1) index(a 계약_x1) index(c 가입부가상품_PK) index (d 상품PK)*/
```

[ordered 힌트 사용시]

```
/*+ ordered use_hash(b) use_nl(c) use_hash(d) swap_join_inputs(b)  
swap_join_inputs(d)  
Index(b 가입상품_X1) index(a 계약_x1) index(c 가입부가상품_PK) index (d 상품PK)*/
```

25. 아래는 튜닝이 필요한 SQL에서 스칼라 서브쿼리 부분만 발췌한 것이다. 스칼라 서브쿼리를 수행하는 부분에서 현재의 문제점을 기술하고, 튜닝 방안을 제시하시오

```
SELECT ...  
  , (SELECT CASE WHEN A. 일할계산여부 = 'Y'  
                THEN NVL(A. 총청구건수, 0) - NVL(A. 청구횟수, 0)  
                ELSE B. 할부개월수 - NVL(A. 청구횟수, 0)  
        END  
    FROM 서비스별할부 A, 할부계획 B  
   WHERE A. 서비스계약번호 = AA.서비스계약번호  
        AND A. 할부상태코드 = '01'  
        AND B. 할부계획ID(+) = A.할부계획ID  
        AND ROWNUM <= 1  
   ) AS 청구횟수, ...  
FROM ( ... ) AA  
WHERE ...
```

문제 및 풀이

Id	Operation	Name	Starts	A-Rows	A-Time	Buffers
...	...	...	...	...	...	...
26	COUNT STOPKEY		37243	28953	00:00:34.56	293K
27	NESTED LOOP OUTER		37243	28953	00:00:34.56	293K
28	TABLE ACCESS BY INDEX ROWID	서비스별할부	37243	28953	00:00:17.26	177K
29	INDEX RANGE SCAN	서비스별할부_X1	37243	28953	00:00:08.05	149K
30	TABLE ACCESS BY INDEX ROWID	할부계획	28953	28953	00:00:17.26	115K
31	INDEX UNIQUE SCAN	할부계획_PK	28953	28953	00:00:08.15	86860
...	...	...	...	...	...	...